

Agent Router + Kyverno Authz: Live Governance for Models and Tools

Deepak Goel, Nutanix | Jim Bugwadia, Nirmata

KyvernoCon | Cloud Native Theater | Tue, Nov 10, 4:30 PM

Who we are

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER



Deepak Goel
CTO, Cloud Native
Nutanix



Jim Bugwadia
Co-founder and CEO
Nirmata

Your agent needs a router

Start with Agent Router: one integration for models and tools

Without a shared router

Provider SDKs and formats

Credentials in applications

Retries and fallback logic

Separate MCP integrations

With Agent Router

A familiar model API and MCP endpoint

Platform-managed credentials

Declarative routes and failover

Central limits and telemetry

Agent Router controls; Envoy carries

Open source, powered by Envoy and Envoy Gateway

Agent Router declares intent

Logical models and provider mapping

MCP server aggregation

Routes and backend credentials

Quotas and usage attribution

Envoy carries traffic

Request routing and retries

Authentication and rate limits

Circuit breaking and telemetry

Extension points for policy

One contract, more models and tools

Keep provider and tool integration logic out of the agent

Models: /v1

Use an OpenAI-compatible client

Request a logical model name

Map to provider-specific models

Keep provider keys behind the router

Tools: /mcp

Connect to one MCP endpoint

Aggregate multiple MCP servers

Select exposed tools per route

Authenticate and authorize calls

Native governance is already valuable

Kyverno Authz extends the platform; it does not replace it

Already built in

JWT authentication

Static MCP tool selection

MCP CEL request allow/deny

Quotas, limits and access telemetry

The next requirements

Use current Kubernetes permissions

Change a response per user

Govern model request bodies

Manage exceptions and policy reports

Kyverno Authz: live policy at the gateway

Kyverno Con 2026 NORTH AMERICA CLOUD NATIVE THEATER

Authorize business intent with reusable CEL policies

Request authorization

ext_authz: allow or deny

Read MCP and model request bodies

Call Kubernetes SubjectAccessReview

Inject verified identity headers

Response governance

ext_proc prototype: inspect and rewrite

Filter MCP tools/list per user

Keep execution authorization in place

Independent policy service lifecycle

Native checks vs. Kyverno Authz

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

Use the simplest control that satisfies the requirement

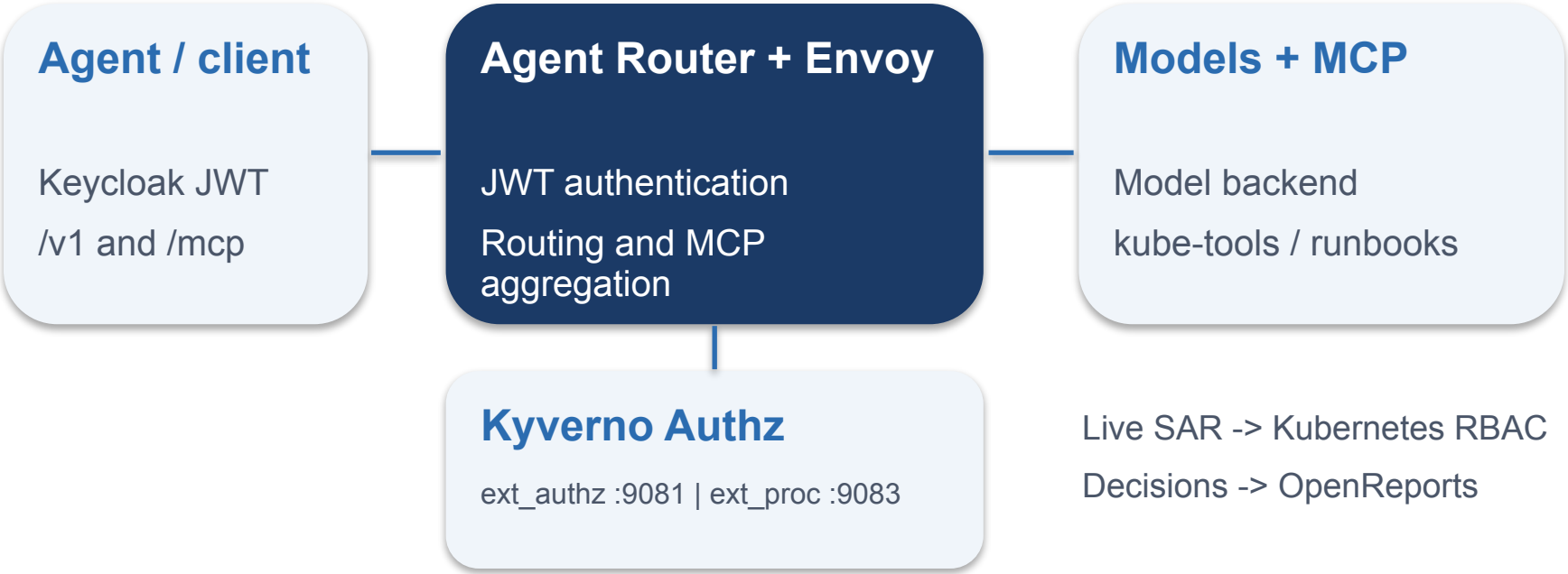
Tool allowlists + replica bounds	**Native MCP CEL can do this**
Current Kubernetes RBAC	**Kyverno Authz: live SAR lookup**
Per-user tools/list response	**Kyverno Authz: ext_proc prototype**
Model-body checks + tenant header	**Kyverno Authz: ext_authz policy**
Exceptions + decision reports	**Kyverno Authz lifecycle and OpenReports**

Kyverno admission is complementary platform governance, not runtime Authz.

Reference architecture

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

Agent Router routes; Kyverno Authz evaluates; Envoy enforces



Kyverno admission + cleanup govern routes and the exception carrier.

Use case: recover checkout safely

A deterministic operations agent workflow on local Kubernetes

Actors and resources

Alice: team-a-dev / checkout

Bob: team-b-dev / checkout

Tools: status, events, logs, scale

Scale is bounded to 1-3 replicas

Questions policy must answer

Can this user act right now?

Should this tool be discoverable?

Is this model request acceptable?

Is emergency access scoped + recorded?

Demo: seven verified scenarios

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

Local Kind cluster: kyverno-agentrouter

1 Live RBAC + tool discovery

2 Shared request guardrails

3 Model request governance

4 Incident-scoped exception

5 OpenReports decisions

6 Complementary admission

7 Fail-closed integration

All seven scenarios passed on October 19, 2026; not rerun for this deck.

Demo 1: same token; different live permissions

The strongest moment: discovery AND execution follow current RBAC

```
RBAC revoked:  Alice -> 4 tools; scale hidden
Direct scale:  HTTP 403; no backend execution
Bob:           5 tools; normal access preserved
RBAC restored: Alice -> 5 tools; scale visible
Same JWT and MCP session; scale succeeds
```

No new token. No Agent Router route change. A live SAR changes the decision.

How the policies see intent

Use the real JSON-RPC body, verified identity and live Kubernetes state

Request policy: Envoy mode

JWT -> preferred_username

JSON-RPC -> method / tool / args

SAR -> current resource permission

Return envoy.Allowed / Denied

Processor policy: prototype

Response phase + tools/list request

Single SSE data payload -> JSON

SAR -> filter result.tools

extproc.ReplaceBody -> reframe SSE

Demo 2-3: one language for tools and models

Be honest: some request checks are possible with native MCP CEL

Tool guardrails

Exact workload + namespace

Known tools and arguments

Integer replica count: 1-3

Reject before backend execution

Model-body governance

Alice: demo-mini; Bob: demo-pro

Bound max_tokens; n must be 1

Restrict function calling

Block credential patterns in prompts

Demo 4: scoped incident access

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

Bob recovers team A only within the INC-1234 exception

```
No exception:           cross-team scale -> DENY
Activate + reload:      Bob + INC-1234 -> ALLOW
No incident header:     DENY
Wrong user / tool:      DENY
Replicas out of bounds: DENY
Revoke + reload:        cross-team scale -> DENY
```

The exception skips the whole policy, including SAR: scope every condition.

Manage exceptions, not just bypasses

Govern the exception carrier and retain a hard request-time expiry

Complementary admission

Require incident metadata

Require narrow exception scope

Restrict referenced policy names

Cap configured TTL at one hour

Runtime + lifecycle

15-minute demo expiry

Cleanup deletes the ConfigMap

Explicit rollout loads / revokes

Expired requests stop matching

Demo 5: report decisions, not credentials

OpenReports connects a request to the policy that decided

```
RESULT | fail | agentrouter-bounded-operations
RESULT | pass | agentrouter-model-governance
RESULT | pass | (default allow) [exception]

Source: kyverno-authz
Properties: status, reason, method, path, host
Sensitive headers: [REDACTED]; bodies omitted
```

Exception passes are unattributed; ext_proc decisions are not reported yet.

Demo 6-7: platform guards + fail closed

Kyverno Con 2026 NORTH AMERICA CLOUD NATIVE THEATER

Separate Kyverno admission from runtime Kyverno Authz

Kyverno admission + cleanup

Reject unguarded routes

Require JWT + body forwarding

Reject fail-open / inline secrets

Govern exception carrier metadata

Envoy + Kyverno Authz

Reject bad or missing tokens

Reject malformed / oversized bodies

Deny before backend execution

Fail closed on authorizer outage

Run it locally

No cloud model credentials required

```
./examples/agentrouter/scripts/setup.sh
./examples/agentrouter/scripts/run.sh
./examples/agentrouter/scripts/run.sh --check
./examples/agentrouter/scripts/reset.sh
```

Real gateway, JWTs, MCP sessions, Kubernetes RBAC and deployment changes.

Deterministic MCP/model backends make the results repeatable.

Prototype limits and deployment gaps

A working demo is not production readiness

Processing + reporting

ext_proc is a local prototype

Buffered bodies add latency + memory

Compressed / multi-chunk bypass

No ext_proc decision reports yet

Policy deployment

Kyverno self-validation conflicts

File policy + exception fallback

Exception reload rolls authorizer

No automated approval workflow

Route once; govern with live context

A complementary stack, not a replacement gateway

Agent Router

One integration for models and tools

Keep secrets behind the router

Declare routing and capacity intent

Envoy enforces the traffic path

Kyverno Authz

Authorize with live external state

Govern requests and responses

Reuse versioned CEL policies

Manage exceptions + decision reports

References and demo sources

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

Agent Router foundation; Kyverno Authz complementary policy

Agent Router project + documentation

https://github.com/theagentrouter/agent-router

Your Agents Need a Router - Ignasi Barrera

https://agntconmcpconeu26.sched.com/event/2WF5u

Demo guide + policy sources

examples/agentrouter/README.md

Implementation: ext_authz + ext_proc prototype

pkg/authz/envoy/ and pkg/authz/extproc/